RNNCell#

https://pytorch.org/docs/stable/generated/torch.nn.modules.rnn.RNNCell.html

Description:

An Elman RNN cell with tanh or ReLU non-linearity. If nonlinearity is ‘relu’, then ReLU is used in place of tanh. input_size (int) – The number of expected features in the input x hidden_size (int) – The number of features in the hidden state h bias (bool) – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

Function Signature

```
class torch.nn.modules.rnn.RNNCell(input_size, hidden_size, bias=True, nonlinearity='tanh', device=None, dtype=None) [source]#

Parameters

Inputs: input, hidden

Outputs: h’

Shape:
```

Code Examples

```
>>> rnn = nn.RNNCell(10, 20)
>>> input = torch.randn(6, 3, 10)
>>> hx = torch.randn(3, 20)
>>> output = []
>>> for i in range(6):
...     hx = rnn(input[i], hx)
...     output.append(hx)
```

Notes & Warnings

NOTE

Note All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1

Detailed Documentation

Documentation

An Elman RNN cell with tanh or ReLU non-linearity.

If nonlinearity is ‘relu’, then ReLU is used in place of tanh.

input_size (int) – The number of expected features in the input x

hidden_size (int) – The number of features in the hidden state h

bias (bool) – If False, then the layer does not use bias weights b_ih and b_hh. Default: True

nonlinearity (str) – The non-linearity to use. Can be either 'tanh' or 'relu'. Default: 'tanh'

input: tensor containing input features

hidden: tensor containing the initial hidden state Defaults to zero if not provided.

h’ of shape (batch, hidden_size): tensor containing the next hidden state for each element in the batch

input: (N,Hin)(N, H_{in})(N,Hin) or (Hin)(H_{in})(Hin) tensor containing input features where HinH_{in}Hin = input_size.

hidden: (N,Hout)(N, H_{out})(N,Hout) or (Hout)(H_{out})(Hout) tensor containing the initial hidden state where HoutH_{out}Hout = hidden_size. Defaults to zero if not provided.

output: (N,Hout)(N, H_{out})(N,Hout) or (Hout)(H_{out})(Hout) tensor containing the next hidden state.

weight_ih (torch.Tensor) – the learnable input-hidden weights, of shape (hidden_size, input_size)

weight_hh (torch.Tensor) – the learnable hidden-hidden weights, of shape (hidden_size, hidden_size)

bias_ih – the learnable input-hidden bias, of shape (hidden_size)

bias_hh – the learnable hidden-hidden bias, of shape (hidden_size)

All the weights and biases are initialized from U(−k,k)\mathcal{U}(-\sqrt{k}, \sqrt{k})U(−k ,k) where k=1hidden_sizek = \frac{1}{\text{hidden_size}}k=hidden_size1
